



SMART GREENHOUSE DATA HUB

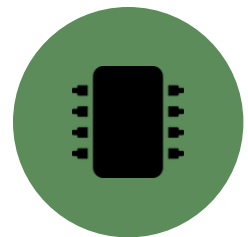
Verbindung von Mikrocontroller-Sensorik und Einplatinencomputer-Datenverarbeitung

[4FSC0SD002.1] Produktion · [4SAE0SD001.1] Produktion

Pitchpräsentation

Das Konzept

Zwei Module, ein durchgängiges System



Modul 1 — Arduino

Sensorik & Aktorik im Gewächshaus

- Temperatur- & Feuchtigkeitssensor (Input)
- Wasserpumpe / Lüfter (Aktor)
- UART-Kommunikation zum Raspberry Pi

misst die Umgebung und reagiert automatisch



Modul 2 — Raspberry Pi

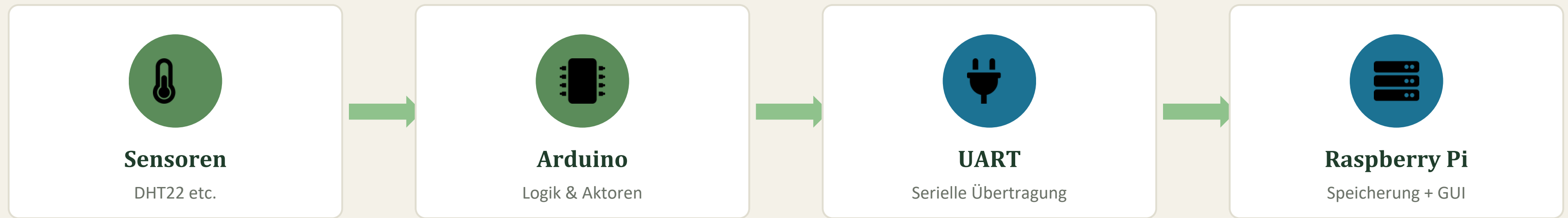
Data Hub & GUI

- Serielle Datenerfassung (Gerätezugriff)
- Data Collector als systemd-Service
- Native GUI mit Live-Diagrammen & Verlauf

speichert, verarbeitet und visualisiert die Daten

System-Architektur

Vom Sensor bis zur Visualisierung



Auf dem Raspberry Pi:



Speicherung in SQLite



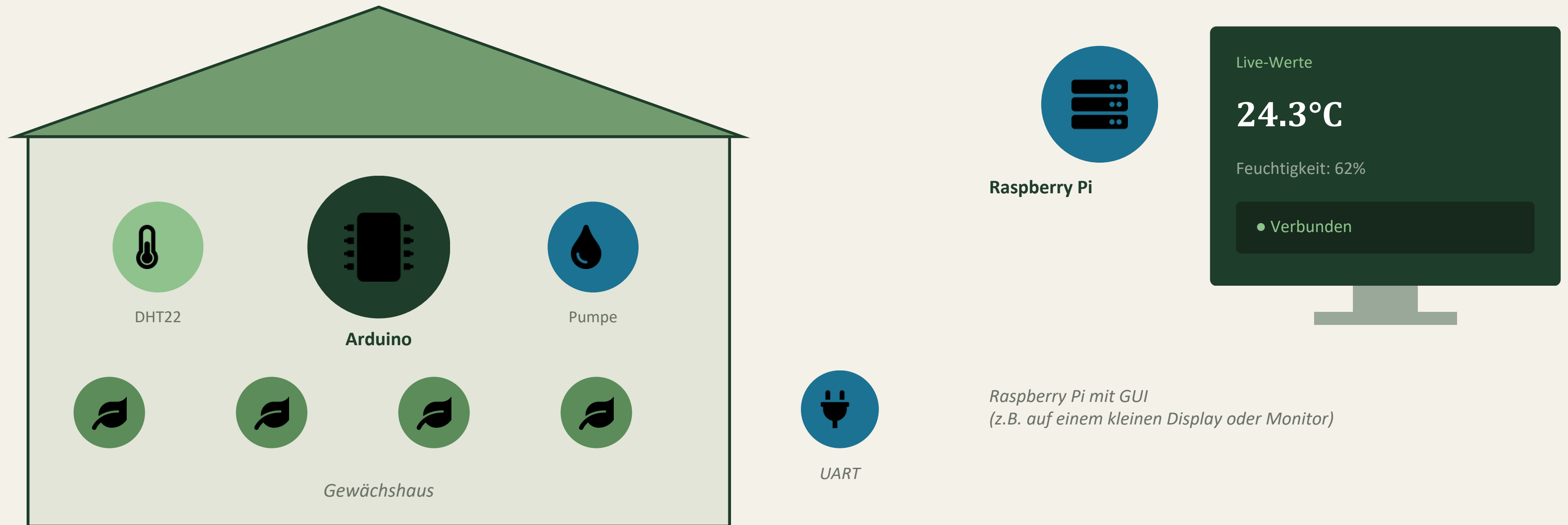
Live-Diagramme (GUI)



systemd Hintergrunddienst

Physischer Aufbau

So könnte die Installation im Gewächshaus aussehen



Hinweis: Illustrative Darstellung — die tatsächliche Platzierung hängt von der Gewächshausgröße und Kabellänge ab.

Modul 1 — Arduino Hardwarekomponenten

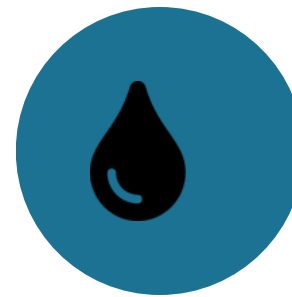
3 Komponenten aus 3 verschiedenen Kategorien (Anforderung: min. 2)



INPUT

**Temperatur- &
Feuchtigkeitssensor**

z.B. DHT22 — erfasst die
Umgebungsbedingungen im Gewächshaus



AKTOREN

Wasserpumpe / Lüfter

reagiert automatisch, wenn Schwellenwerte
über-/unterschritten werden



KOMMUNIKATION

UART

sendet Messwerte kontinuierlich an den
Raspberry Pi

Modul 2 — Betriebssystem-Architektur

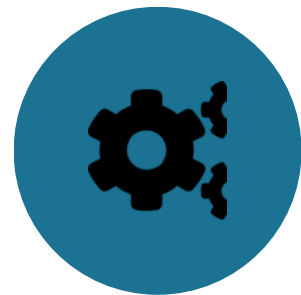
2 Komponenten aus 2 verschiedenen Kategorien (Anforderung: min. 2)



GERÄTEZUGRIFF

Serielle Schnittstelle (/dev/ttyUSB0)

Der Data Collector liest Sensordaten direkt vom Arduino über den seriellen Geräteknoten des Betriebssystems.



PROZESS- UND SYSTEMVERWALTUNG

systemd-Service

Der Data Collector läuft als Hintergrunddienst — startet automatisch beim Booten und wird bei einem Absturz neu gestartet.

Die GUI

Native Oberfläche mit Rust + egui



Live-Diagramme

Temperatur, Feuchtigkeit & mehr in Echtzeit



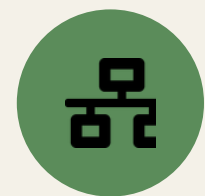
Verlaufsansicht

Historische Daten aus der SQLite-Datenbank



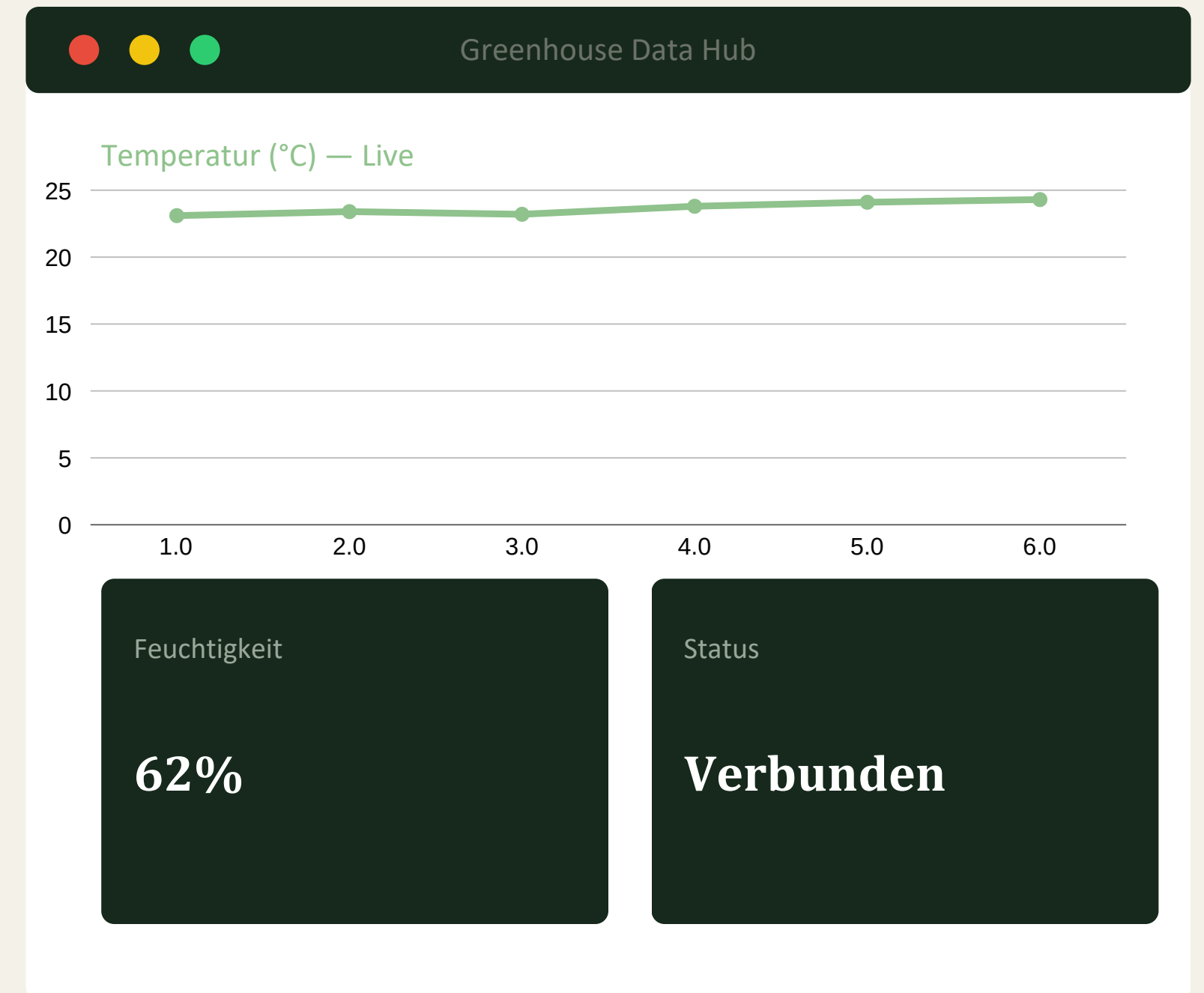
Schwellenwert-Alarme

Warnung bei kritischen Messwerten



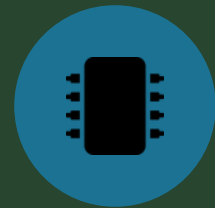
Verbindungsstatus

Zeigt Live, ob Arduino verbunden ist



Tech Stack

Werkzeuge für ein robustes, natives System



Rust

Speichersicherheit & Performance



egui + eframe

Natives GUI-Framework



egui_plot

Live-Diagramme in der GUI



serialport crate

UART-Kommunikation mit Arduino



SQLite (rusqlite)

Lokale Datenspeicherung

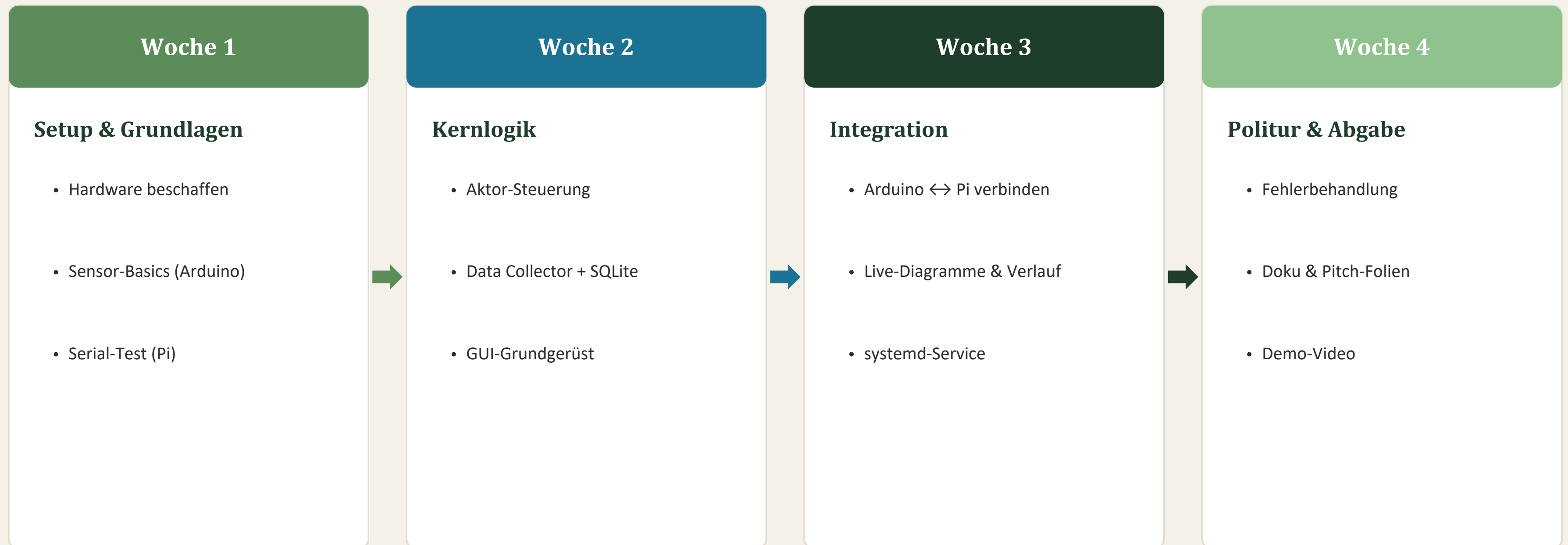


systemd

Hintergrunddienst-Verwaltung

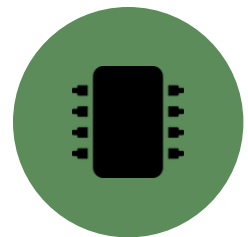
Zeitplan

4 Wochen, parallele Arbeit zu zweit



Aufgabenteilung im Team

Parallele Entwicklung — verbunden über ein gemeinsames Datenformat



Teammitglied 1

Arduino / Hardware

- Sensor- & Aktor-Verkabelung
- Steuerlogik (Schwellenwerte)
- Serielle Datenübertragung
- Hardware-Debugging



Teammitglied 2

Raspberry Pi / GUI

- Rust-Projekt & Serial-Reader
- SQLite-Datenbank
- GUI-Entwicklung (egui)
- systemd-Service

Weitere betrachtete Ideen

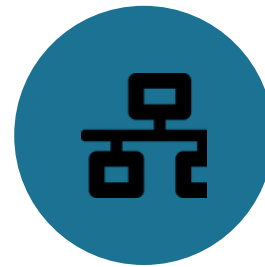
Auch geprüft, aber der Greenhouse Data Hub passte am besten



Security Camera Monitor

Gerätezugriff + Netzwerk-Features

USB-/Pi-Kamera erfasst Bewegungen, GUI zeigt Live-Bild und Ereignis-Log; Stream läuft über das Netzwerk.



Home Network Guard

Netzwerk-Features + Sicherheit

Scannt Geräte im lokalen Netzwerk und erkennt unbekannte Verbindungen; GUI listet aktive Geräte mit Warnungen.



Backup & Sync Station

Dateisystemfeatures + Paketmanagement

Automatisierte Datensicherung mit Zeitplan; GUI verwaltet Speicherplatz, Wiederherstellung und Backup-Verlauf.

Warum dieses Projekt?



Zwei Produktionen zu einem durchgängigen System verbunden



Erfüllt beide Anforderungskataloge (Mikrocontroller + OS-Architektur)



Baut auf vorhandener Erfahrung mit Datenpipelines & Monitoring auf

Vielen Dank — Fragen?